

Lekcja

Temat: Modele baz danych

Na potrzeby baz danych zostały zdefiniowane klasyczne techniki organizowania informacji, zwane modelami baz danych.



Model danych to abstrakcyjny opis sposobu przedstawiania i wykorzystania danych. Definiuje on logiczną reprezentację danych oraz relacje między nimi.

Na model danych składają się:

- struktura — opis sposobu przedstawiania obiektów (encji) modelowanego wycinka świata oraz ich związków,
- ograniczenia — reguły kontrolujące spójność i poprawność danych,
- operacje — zbiór działań, które umożliwiają dostęp do struktur.

Głównymi modelami baz danych są:



1. Tradycyjne (przedrelacyjne) modele

- a) **model hierarchiczny,**
- b) **model sieciowy,**



2. **Model relacyjny** (najpopularniejszy od lat 80.)



3. **Modele post-relacyjne / NoSQL**

a) **Model dokumentowy**

- Dane przechowywane jako dokumenty (JSON, BSON, XML)
- Struktura hierarchiczna wewnątrz dokumentu
- **Przykład:** MongoDB, Couchbase
- **Użycie:** katalogi produktów, profile użytkowników

b) **Model klucz-wartość**

- Proste pary: klucz → wartość (dowolne dane)
- Bardzo szybki dostęp po kluczu
- **Przykład:** Redis, Amazon DynamoDB
- **Użycie:** cache, sesje, konfiguracje

c) **Model szerokokolumnowy (kolumnowy)**

- Dane przechowywane w kolumnach (nie w wierszach)
- Optymalny dla agregacji i analiz
- **Przykład:** Cassandra, HBase, Google Bigtable
- **Użycie:** analityka, big data, systemy czasowych szeregów

d) **Model grafowy**

- Dane jako węzły, krawędzie i właściwości

- Idealny dla powiązań i relacji
- **Przykład:** Neo4j, Amazon Neptune
- **Użycie:** sieci społecznościowe, rekomendacje, wykrywanie oszustw



4. Nowoczesne / hybrydowe modele

a) Modele wielomodelowe

- Łączą różne modele w jednym systemie
- **Przykład:** PostgreSQL (relacyjny + JSON + graf), ArangoDB (dokumentowy + grafowy)

b) Bazy time-series

- Zoptymalizowane pod dane czasowe (znacznik czasu jako główny wymiar)
- **Przykład:** InfluxDB, TimescaleDB

c) Bazy przestrzenne/geograficzne

- Obsługa danych geograficznych i przestrzennych
- **Przykład:** PostGIS (rozszerzenie PostgreSQL)

d) Bazy pamięciowe (in-memory)

- Cała baza w pamięci RAM
- **Przykład:** Redis, MemSQL, SAP HANA



5. Model obiektowy i obiektowo-relacyjny



Model hierarchiczny

To jeden z najstarszych modeli baz danych (powstał w latach 60. XX wieku), w którym dane są organizowane w strukturę **drzewa** (hierarchii). Każde "drzewo" składa się z **węzłów**:

- **Jeden węzeł główny** (korzeń) – nie ma rodzica.
- **Węzły potomne** – każdy ma dokładnie **jednego rodzica**, ale może mieć wiele dzieci.
- Relacje są typu **jeden-do-wielu (1:N)**.
- Dostęp do danych często odbywa się poprzez ścieżki od korzenia do konkretnego rekordu.

Przykład modelu hierarchicznego: Struktura Technikum

```
Technikum (korzeń)
|
|— Kierunek: Technikum Informatyczne
|   |
|   |— Przedmiot: Matematyka
|   |— Przedmiot: Język polski
|   |
|   |— Uczeń: Nowak Tadeusz
|
|— Kierunek: Technikum Programistyczne
|   |
|   |— Przedmiot: Matematyka
|   |— Przedmiot: Język polski
|   |
|   |— Uczeń: Kowalski Jan
```

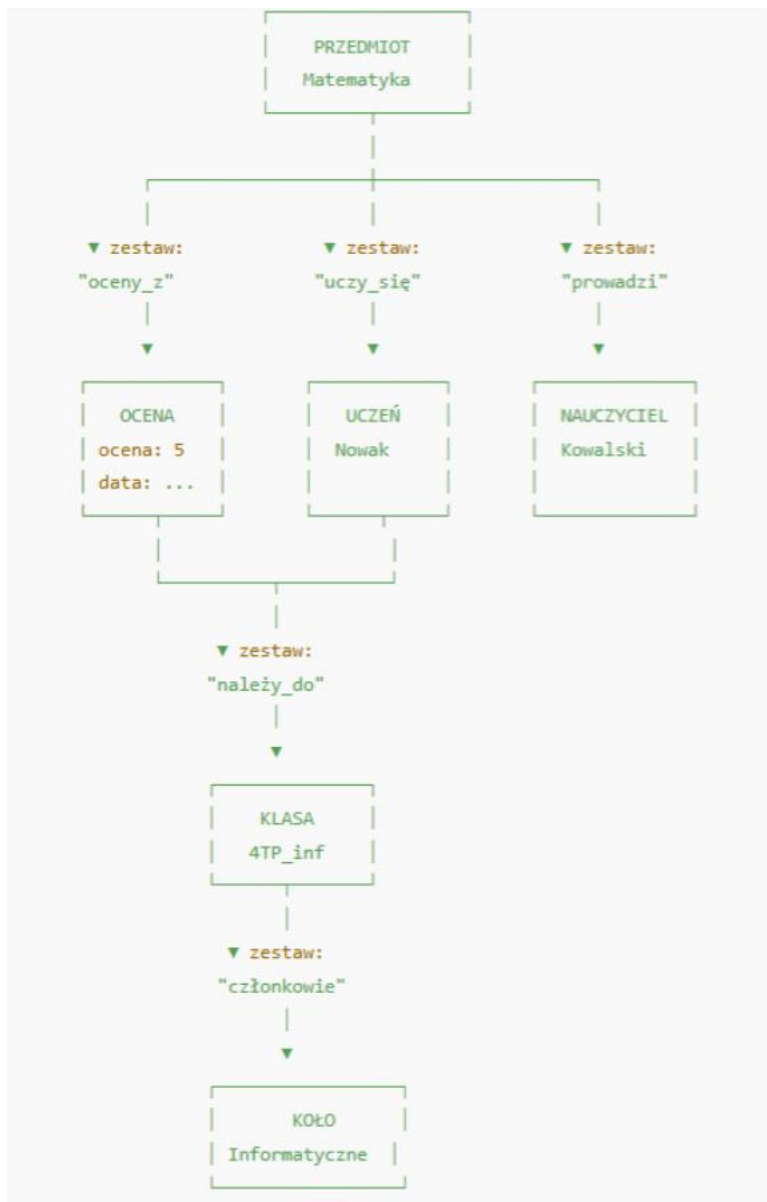
Model sieciowy

Model sieciowy powstał jako rozwinięcie modelu hierarchicznego, aby rozwiązać jego główne ograniczenia. Został sformalizowany przez grupę **CODASYL** (Conference on Data Systems Languages) w latach 60-70.

Kluczowe cechy:

1. **Struktura grafu** – dane organizowane są jako zbiór rekordów połączonych w dowolną strukturę grafową (nie tylko drzewo)
2. **Jeden rekord może mieć wielu rodziców** – w przeciwieństwie do hierarchicznego (tylko jeden rodzic)
3. **Relacje przechowywane jako wskaźniki fizyczne** między rekordami
4. **Dwa podstawowe elementy:**
 - a. **Typ rekordu** – odpowiednik tabeli w modelu relacyjnym
 - b. **Zbiór (set)** – nazwana relacja łącząca typy rekordów
5. **Obsługa relacji wiele-do-wielu (M:N)** – główna przewaga nad modelem hierarchicznym

Przykład modelu sieciowego: System szkolny



Model relacyjny

Model relacyjny to najpopularniejszy współczesny model baz danych, stworzony przez Edgara F. Codd'a w 1970 roku. Opiera się na matematycznej teorii mnogości i algebrze relacji.

Podstawowe pojęcia:

1. **Relacja (tabela)** – zbiór krotek o takiej samej strukturze
2. **Krotka (wiersz, rekord)** – pojedynczy wpis w tabeli
3. **Atrybut (kolumna, pole)** – nazwane pole z określonym typem danych
4. **Klucz główny (Primary Key)** – unikalny identyfikator wiersza
5. **Klucz obcy (Foreign Key)** – odwołanie do klucza głównego innej tabeli
6. **Domeny** – zbiór dopuszczalnych wartości dla atrybutu

Przykład modelu relacyjnego: System szkolny

1. Tabele i ich struktura:

Tabela UCZNIOWIE:

ID_ucznia	Imię	Nazwisko	ID_klasy
1	Jan	Kowalski	101
2	Anna	Nowak	101
3	Tomasz	Wiśniewski	102

PK: ID_ucznia

FK: ID_klasy → KLASY(ID_klasy)

Tabela KLASY:

ID_klasy	Nazwa_klasy	Rok_szkolny
101	4TI	2024
102	3TP	2024
103	2TE	2024

PK: ID_klasy

Tabela PRZEDMIOTY:

ID_przedmiotu	Nazwa_przedmiotu
1	Matematyka
2	Język polski
3	Bazy danych

PK: ID przedmiotu

Tabela OCENY:

ID_oceny	ID_ucznia	ID_przedmiotu	Ocena	Data
1	1	1	5	2024-01-15
2	1	2	4	2024-01-16
3	2	1	3	2024-01-15
4	3	3	5	2024-01-17

PK: ID_oceny

FK: ID_ucznia → UCZNIOWIE(ID_ucznia)

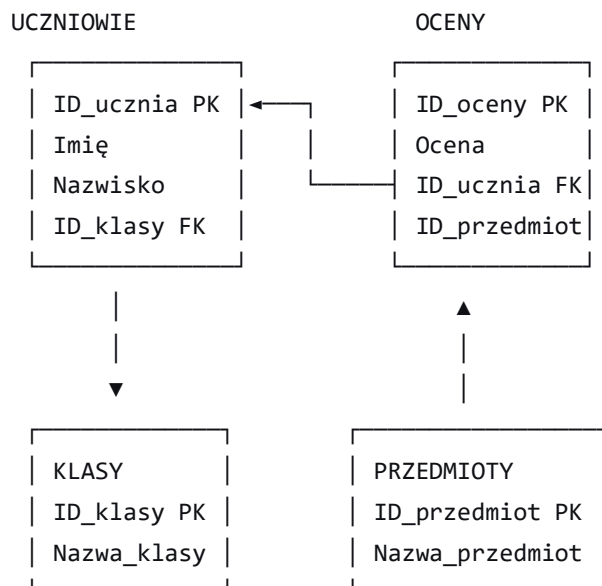
FK: ID_przedmiotu → PRZEDMIOTY(ID_przedmiotu)

Tabela UCZNIOWIE_PRZEDMIOTY (relacja wiele-do-wielu):

ID_ucznia	ID_przedmiotu
1	1
1	2
2	1
3	3

PK: (ID_ucznia, ID_przedmiotu) - klucz złożony

2. Relacje między tabelami (diagram ER):



Rodzaje relacji:

- **1:N** (jeden do wielu) – jedna klasa ma wielu uczniów
- **M:N** (wiele do wielu) – uczeń ma wiele przedmiotów, przedmiot ma wielu uczniów
- **1:1** (jeden do jednego) – rzadziej stosowane

```
CREATE TABLE UCZNIOWIE (  
  ID_ucznia INT PRIMARY KEY,  
  Imię VARCHAR(50),  
  Nazwisko VARCHAR(50),  
  ID_klasy INT,  
  FOREIGN KEY (ID_klasy) REFERENCES KLASY(ID_klasy)  
);  
  
CREATE TABLE OCENY (  
  ID_oceny INT PRIMARY KEY,  
  ID_ucznia INT,  
  ID_przedmiotu INT,  
  Ocena INT CHECK (Ocena BETWEEN 1 AND 6),  
  Data DATE,  
  FOREIGN KEY (ID_ucznia) REFERENCES UCZNIOWIE(ID_ucznia),  
  FOREIGN KEY (ID_przedmiotu) REFERENCES PRZEDMIOTY(ID_przedmiotu)  
);
```

Model dokumentowy

Dane przechowywane jako **dokumenty** w formatach JSON, BSON, XML. Każdy dokument jest samodzielną jednostką zawierającą wszystkie dane o danym obiekcie.

Kluczowe cechy:

- **Samo opisujące dokumenty** – struktura zawarta w dokumencie
- **Dynamiczny schemat** – różne dokumenty mogą mieć różne pola
- **Zagnieżdżanie** – możliwość przechowywania obiektów w obiektach
- **Indeksowanie** – po dowolnych polach dokumentu

Przykład modelu dokumentowego: System blogowy (MongoDB)

```
// Kolekcja "posts"  
{  
  "_id": "507f1f77bcf86cd799439011",  
  "title": "Wprowadzenie do NoSQL",  
  "author": {
```

```

    "name": "Jan Kowalski",
    "email": "jan@example.com",
    "bio": "Ekspert baz danych"
  },
  "tags": ["NoSQL", "MongoDB", "Bazy danych"],
  "content": "NoSQL to nowoczesne podejście...",
  "comments": [
    {
      "user": "Anna Nowak",
      "text": "Świetny artykuł!",
      "date": "2024-01-15",
      "likes": 5
    },
    {
      "user": "Tomasz Wiśniewski",
      "text": "Brakuje przykładów",
      "date": "2024-01-16"
      // Brak pola 'likes' - to OK w modelu dokumentowym!
    }
  ],
  "metadata": {
    "views": 1250,
    "reading_time": "5 min",
    "published": true
  }
}

// Inny post może mieć zupełnie inną strukturę:
{
  "_id": "507f1f77bcf86cd799439012",
  "title": "Nowości w JavaScript",
  "author": "Maria Zielińska", // String zamiast obiektu!
  "category": "programming",
  "body": "ES2024 wprowadza...", // Inna nazwa pola niż 'content'
  "created_at": "2024-01-20T10:30:00Z"
}

```

Model klucz-wartość

Najprostszy model – jak **słownik** lub **hashmapa**. Klucz (unikalny identyfikator) → Wartość (dowolne dane).

Kluczowe cechy:

- **Ekstremalnie szybki dostęp** $O(1)$ po kluczu
- **Brak schematu** – wartość może być czymkolwiek
- **Proste operacje:** GET, SET, DELETE
- **TTL (Time To Live)** – automatyczne usuwanie po czasie

Przykład modelu klucz-wartość: System cache i sesji (Redis)

```
# Cache stron internetowych
SET "page:/products/laptop" "<html>...duża strona...</html>"
EXPIRE "page:/products/laptop" 300 # Usun po 5 minutach

# Koszyk zakupowy użytkownika
HSET "cart:user123" "laptop" 2 "phone" 1 "headphones" 1
EXPIRE "cart:user123" 3600 # Wygaśnięcie koszyka po 1 godzinę

# Licznik odwiedzin
INCR "page_views:/homepage"
INCRBY "page_views:/homepage" 5

# Leaderboard gry
ZADD "game_leaderboard" 1500 "player1" 2300 "player2" 1800 "player3"
ZREVRANGE "game_leaderboard" 0 2 WITHSCORES # Top 3 graczy

# Sesja użytkownika
SET "session:abc123" '{"user_id": 456, "username": "janek", "role": "admin"}'
EXPIRE "session:abc123" 1800 # Sesja wygasa po 30 minut
```

Model szerokokolumnowy (kolumnowy)

Dane przechowywane w **kolumnach** zamiast wierszy. Każda kolumna przechowywana osobno, optymalna dla agregacji.

Kluczowe cechy:

- **Przechowywanie kolumnowe** – szybkie skanowanie kolumn
- **Skalowanie poziome** – partycjonowanie
- **Zoptymalizowany dla zapytań agregujących**
- **Tolerancja na brak spójności** (eventual consistency)

Przykład modelu szerokokolumnowy: Dane sensorów IoT (Cassandra)

```
-- Tworzenie tabeli (Cassandra Query Language - podobne do SQL)
CREATE TABLE sensor_readings (
    sensor_id uuid,
    date date,
    timestamp timestamp,
    temperature decimal,
    humidity decimal,
    pressure decimal,
    location text,
    PRIMARY KEY ((sensor_id, date), timestamp)
) WITH CLUSTERING ORDER BY (timestamp DESC);
```

```
-- Wstawianie danych
INSERT INTO sensor_readings
(sensor_id, date, timestamp, temperature, humidity, location)
VALUES (
    uuid(),
    '2024-01-20',
    '2024-01-20 10:30:00',
    22.5,
    45.0,
    'Warszawa'
);

-- Struktura przechowywania (kolumnowa):
-- Na dysku dane są grupowane KOLUMNOWO:
-- Wszystkie wartości 'temperature' obok siebie
-- Wszystkie wartości 'humidity' obok siebie

-- Zapytanie: średnia temperatura wg lokalizacji
SELECT location, AVG(temperature) as avg_temp
FROM sensor_readings
WHERE date = '2024-01-20'
GROUP BY location;
```

Model grafowy

Dane jako **węzły** (encje) i **krawędzie** (relacje). Idealny do reprezentowania powiązań.

Kluczowe cechy:

- **Węzły** – encje z właściwościami
- **Krawędzie** – relacje z typem i właściwościami
- **Przechodzenie grafu** – znajdowanie ścieżek
- **Wykrywanie wzorców** – znajdowanie podgrafów

Przykład model grafowy

```
CREATE (jan:User {
    id: 1,
    name: "Jan Kowalski",
    age: 28,
    city: "Warszawa"
})

CREATE (anna:User {
    id: 2,
    name: "Anna Nowak",
    age: 32,
```

```

    city: "Kraków"
  })

CREATE (tomek:User {
  id: 3,
  name: "Tomasz Wiśniewski",
  age: 25,
  city: "Warszawa"
})

// Tworzenie relacji
CREATE (jan)-[:FRIENDS_WITH {since: "2022-05-10"}]->(anna)
CREATE (jan)-[:FRIENDS_WITH {since: "2023-01-15"}]->(tomek)
CREATE (anna)-[:FOLLOWS]->(tomek)

// Jan lubi programowanie
CREATE (java:Skill {name: "Java", category: "programming"})
CREATE (python:Skill {name: "Python", category: "programming"})
CREATE (jan)-[:KNOWS {level: "expert"}]->(java)
CREATE (jan)-[:KNOWS {level: "intermediate"}]->(python)
CREATE (anna)-[:KNOWS {level: "beginner"}]->(python)

// Zapytanie: znajdź znajomych Jana którzy znają Javę
MATCH (jan:User {name: "Jan Kowalski"})-[:FRIENDS_WITH]-(friend:User)
WHERE (friend)-[:KNOWS]->(:Skill {name: "Java"})
RETURN friend.name

// Zapytanie: znajdź ścieżkę między Janem a osobą znającą Python
MATCH path = (jan:User {name: "Jan Kowalski"})-[:*..3]-(pythonExpert:User)
WHERE (pythonExpert)-[:KNOWS]->(:Skill {name: "Python"})
RETURN path

```

Model obiektowy i obiektowo-relacyjny

1. Model obiektowy

Model obiektowy przenosi zasady **programowania obiektowego (OOP)** do bazy danych. Dane przechowywane są jako **obiekty** z:

- **Atrybutami** (pola, właściwości)
- **Metodami** (zachowanie)
- **Dziedziczeniem** (hierarchie klas)
- **Enkapsulacją** (hermetyzacja)

Kluczowe cechy:

- **Obiekty z identyfikatorami OID** (Object ID)
- **Bezpośrednie mapowanie** obiektów aplikacji na obiekty bazy
- **Brak potrzeby ORM** (Object-Relational Mapping)
- **Zachowanie z danymi** – metody przechowywane w bazie

Przykład modelu obiektowego: System biblioteczny (db4o)

```
// Klasy Java (również przechowywane w bazie)
public class Osoba {
    private String id;
    private String imie;
    private String nazwisko;

    public Osoba(String imie, String nazwisko) {
        this.id = UUID.randomUUID().toString();
        this.imie = imie;
        this.nazwisko = nazwisko;
    }

    // Metody - również dostępne w zapytaniach
    public String getPelnaNazwa() {
        return imie + " " + nazwisko;
    }
}

public class Czytelnik extends Osoba {
    private Date dataRejestracji;
    private List<Wypozyczenie> wypozyczenia = new ArrayList<>();

    public Czytelnik(String imie, String nazwisko) {
        super(imie, nazwisko);
        this.dataRejestracji = new Date();
    }

    public void wypożyczKsiazke(Ksiazka ksiazka) {
        Wypozyczenie w = new Wypozyczenie(this, ksiazka);
        wypozyczenia.add(w);
        ksiazka.setWypozyczona(true);
    }

    public int getLiczbaWypozyczen() {
        return wypozyczenia.size();
    }
}

public class Ksiazka {
    private String isbn;
    private String tytul;
    private Autor autor;
    private boolean wypozyczona;

    // Metoda w klasie przechowywanej w bazie
    public boolean jestDostepna() {
        return !wypozyczona;
    }
}

// Operacje na bazie obiektowej (db4o)
ObjectContainer db = Db4oEmbedded.openFile("biblioteka.db");
```

```
// Zapis obiektu - bez mapowania!
Czytelnik czytelnik = new Czytelnik("Jan", "Kowalski");
Ksiazka ksiazka = new Ksiazka("123-456", "Programowanie w Java");
db.store(czytelnik);
db.store(ksiazka);

// Zapytanie natywne - użycie metod obiektów!
List<Czytelnik> czytelnicy = db.query(new Predicate<Czytelnik>() {
    public boolean match(Czytelnik czytelnik) {
        // Używamy metody obiektu w zapytaniu!
        return czytelnik.getLiczbaWypozycczen() > 5;
    }
});

// Zapytanie QBE (Query by Example)
Czytelnik przykład = new Czytelnik(null, "Kowalski");
List<Czytelnik> wynik = db.queryByExample(przykład);

// Pobranie powiązanych obiektów
czytelnik.wypożyczKsiazke(ksiazka); // Użycie metody obiektu
db.store(czytelnik); // Zapis całego grafu obiektów

db.close();
```

Struktura danych w bazie obiektowej:

Obiekt Czytelnik [OID: 0x1234]:

Nagłówek: OID=0x1234, Typ=Czytelnik
Dane: - id: "550e8400-e29b-41d4-a716-446655" - imie: "Jan" - nazwisko: "Kowalski" - dataRejestracji: 2024-01-20 - wypozyczenia: [OID:0x5678, OID:0x9ABC]
Metody (wskazania): - getPelnaNazwa() - wypożyczKsiazke() - getLiczbaWypozycczen()

↓

Obiekt Wypozyczenie [OID: 0x5678]...

Obiekt Ksiazka [OID: 0x9ABC]...

2. Model obiektowo-relacyjny (OR)

Rozszerzenie modelu relacyjnego o cechy obiektowe. Łączy zalety obu światów:

- **Tabele** z relacyjnego

- **Typy złożone**, dziedziczenie, metody z obiektowego

Kluczowe cechy:

1. **UDT** (User-Defined Types) – własne typy danych
2. **Tabele zagnieżdżone** – tabele w tabelach
3. **Dziedziczenie tabel** – hierarchie tabel
4. **Metody w tabelach** – zachowanie w bazie
5. **REF** – referencje do wierszy

Przykład modelu obiektowo-relacyjny: System kadrowy (PostgreSQL z rozszerzeniami OR)

-- PostgreSQL z rozszerzeniami obiektowo-relacyjnymi

-- 1. TWORZENIE TYPÓW ZŁOŻONYCH (UDT)

```
CREATE TYPE adres_typ AS (  
  ulica VARCHAR(100),  
  numer VARCHAR(10),  
  miasto VARCHAR(50),  
  kod_pocztowy VARCHAR(6)  
);
```

```
CREATE TYPE kontakt_typ AS (  
  email VARCHAR(100),  
  telefon VARCHAR(20),  
  telefon_komórkowy VARCHAR(20)  
);
```

-- 2. TABELA Z UDT I METODAMI

```
CREATE TABLE pracownicy (  
  id SERIAL PRIMARY KEY,  
  imie VARCHAR(50),  
  nazwisko VARCHAR(50),  
  adres adres_typ, -- Typ złożony!  
  kontakt kontakt_typ,  
  data_zatrudnienia DATE,  
  wynagrodzenie DECIMAL(10,2),
```

-- Metoda jako funkcja w tabeli

```
FUNCTION pelny_adres() RETURNS VARCHAR  
AS $$  
BEGIN  
  RETURN (adres).ulica || ' ' || (adres).numer || ' ' ||  
    (adres).kod_pocztowy || ' ' || (adres).miasto;  
END;  
$$ LANGUAGE plpgsql  
);
```

-- 3. DZIEDZICZENIE TABEL

```
CREATE TABLE osoby (  
  id SERIAL PRIMARY KEY,
```

```
imie VARCHAR(50),
nazwisko VARCHAR(50),
pesel VARCHAR(11)
);
```

```
CREATE TABLE klienci (
    numer_karty VARCHAR(20),
    data_rejestracji DATE
) INHERITS (osoby); -- Dziedziczenie!
```

```
CREATE TABLE dostawcy (
    nip VARCHAR(10),
    regon VARCHAR(9)
) INHERITS (osoby);
```

-- 4. TABELE ZAGNIEŻDŻONE (kolekcje)

```
CREATE TABLE zamowienia (
    id SERIAL PRIMARY KEY,
    data_zamowienia DATE,
    klient_id INTEGER,

    -- Zagnieżdżona tabela z pozycjami zamówienia
    pozycje_zamowienia pozycja_zamowienia_typ[]
);
```

```
CREATE TYPE pozycja_zamowienia_typ AS (
    produkt_id INTEGER,
    nazwa_produktu VARCHAR(100),
    ilosc INTEGER,
    cena_jednostkowa DECIMAL(10,2),

    -- Metoda w typie
    FUNCTION wartosc_calkowita() RETURNS DECIMAL
    AS $$
    BEGIN
        RETURN $1.ilosc * $1.cena_jednostkowa;
    END;
    $$ LANGUAGE plpgsql
);
```

-- 5. OBIĘKTOWE ZAPYTANIA

-- Wstawianie z typami złożonymi

```
INSERT INTO pracownicy (imie, nazwisko, adres, kontakt)
VALUES (
    'Jan',
    'Kowalski',
    ROW('Marszałkowska', '123', 'Warszawa', '00-001')::adres_typ,
    ROW('jan@firma.pl', '222333444', '555666777')::kontakt_typ
);
```

-- Dostęp do pól typów złożonych

```
SELECT
    imie,
    nazwisko,
```

```

(adres).miasto, -- Dostęp do pola typu złożonego
(kontakt).email,
pelny_adres() -- Wywołanie metody
FROM pracownicy
WHERE (adres).miasto = 'Warszawa';

-- Zapytanie na zagnieżdżonych danych
SELECT
    z.id,
    z.data_zamowienia,
    pz.nazwa_produktu,
    pz.ilosc,
    pz.cena_jednostkowa,
    pz.wartosc_calkowita() -- Metoda na typie zagnieżdżonym
FROM zamowienia z,
    UNNEST(z.pozycje_zamowienia) AS pz -- Rozwijanie zagnieżdżonej tabeli
WHERE pz.wartosc_calkowita() > 1000;

-- 6. REFERENCJE OBIEKTÓW (OID)
CREATE TABLE produkty OF produkt_typ ( -- Tabela obiektów
    PRIMARY KEY (id)
) WITH OIDS; -- Obiektowe identyfikatory

CREATE TABLE zamowienia_ref (
    id SERIAL,
    produkt REF produkt_typ -- Referencja do obiektu
);

-- 7. POLIMORFICZNE FUNKCJE
CREATE OR REPLACE FUNCTION oblicz_podatek(osoba osoby)
RETURNS DECIMAL AS $$
BEGIN
    -- Różne implementacje w zależności od typu
    IF TG_OP = 'klienci' THEN
        RETURN 0; -- Klienci nie płacą podatku
    ELSIF TG_OP = 'dostawcy' THEN
        RETURN 1000; -- Stały podatek
    END IF;
    RETURN 0;
END;
$$ LANGUAGE plpgsql;

```

Lekcja 2

Temat: SQL JOINS w MySQL

```

DROP TABLE Przedmioty;
DROP TABLE Osoby;

```

```

CREATE TABLE Osoby (
    osoba_id INT AUTO_INCREMENT PRIMARY KEY,
    imie VARCHAR(50) NOT NULL,
    nazwisko VARCHAR(50) NOT NULL
);

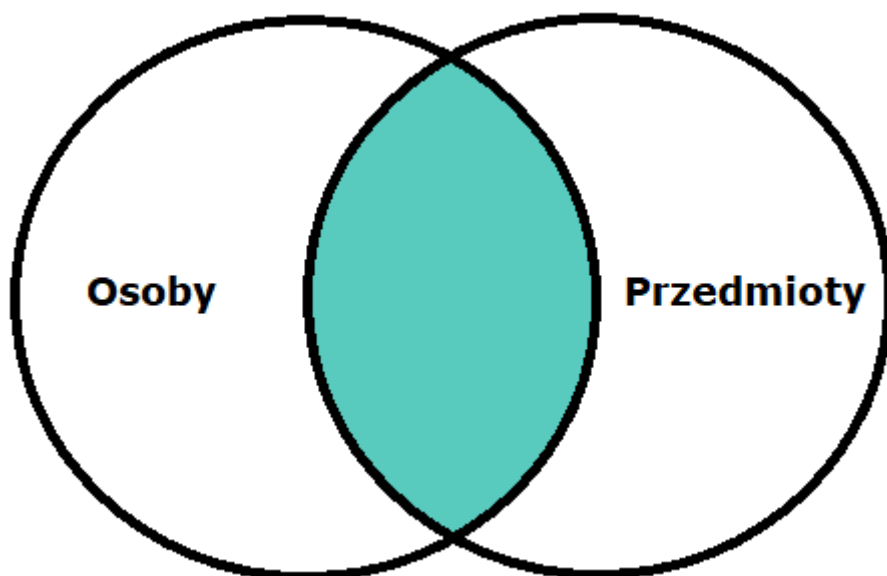
CREATE TABLE Przedmioty (
    przedmiot_id INT AUTO_INCREMENT PRIMARY KEY,
    nazwa VARCHAR(100) NOT NULL,
    osoba_id INT,
    CONSTRAINT fk_przedmiot_osoba FOREIGN KEY (osoba_id) REFERENCES
Osoby(osoba_id)
);

INSERT INTO Osoby (imie, nazwisko) VALUES
('Jan', 'Kowalski'),
('Anna', 'Nowak'),
('Piotr', 'Zieliński'),
('Kasia', 'Wiśniewska'),
('Patryk', 'Nowakowski');

INSERT INTO Przedmioty (nazwa, osoba_id) VALUES
('Laptop', 1),
('Telefon', 1),
('Rower', 2),
('Książka', 3),
('Plecak', 4),
('Kubek', null);

```

INNER JOIN - czyli wszystkie wspólne rekordy, bez NULL



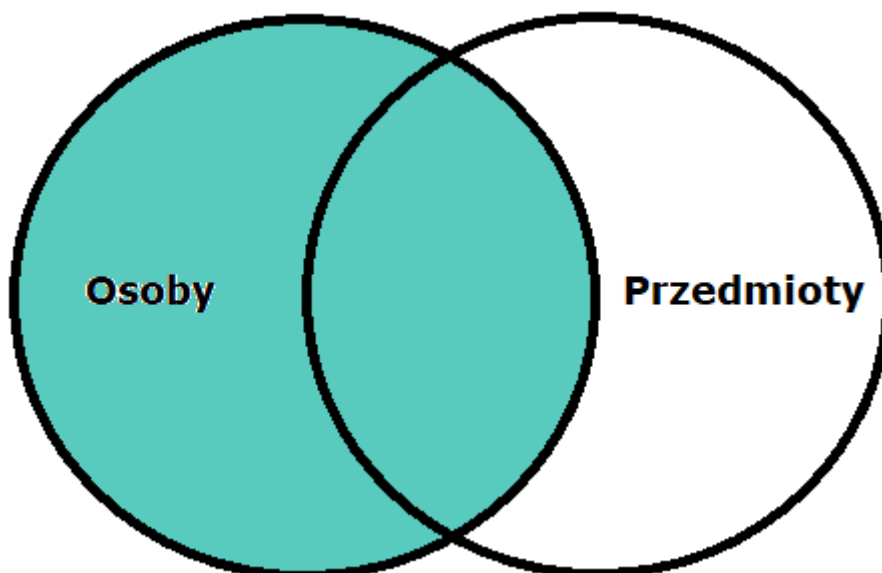
```
select Osoby.imie, Osoby.nazwisko, Przedmioty.nazwa
```

from Osoby

inner join Przedmioty on Osoby.osoba_id = Przedmioty.osoba_id;

imie	nazwisko	nazwa
Jan	Kowalski	Laptop
Jan	Kowalski	Telefon
Anna	Nowak	Rower
Piotr	Zieliński	Książka
Kasia	Wiśniewska	Plecak

LEFT JOIN - czyli wszystkie rekordy z lewej tabeli. W naszym przypadku lewa tabela to Osoby. Jeśli Osoba jest a nie ma dopasowania w tabeli Przedmioty również się wyświetli.

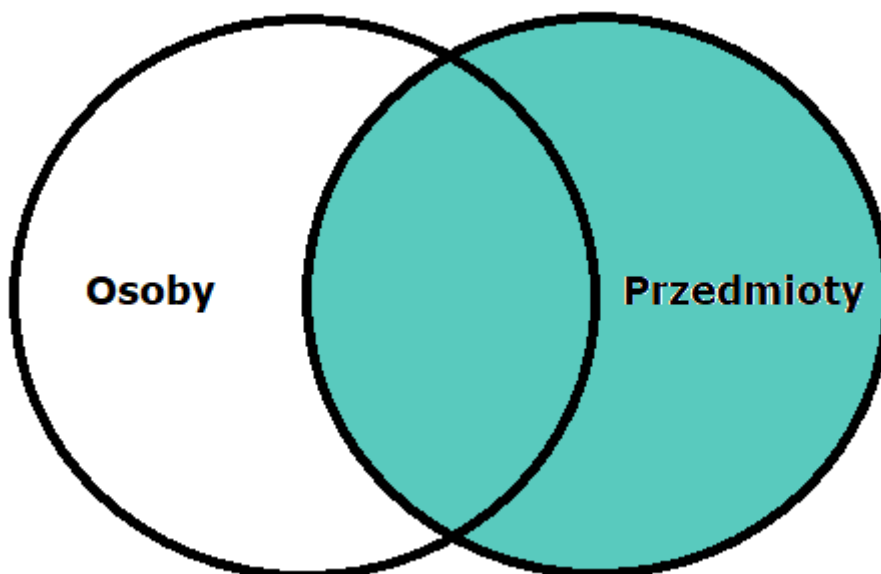


```
SELECT Osoby.imie, Osoby.nazwisko, Przedmioty.nazwa  
FROM Osoby  
LEFT JOIN Przedmioty ON Osoby.osoba_id = Przedmioty.osoba_id;
```

Wynik:

imie	nazwisko	nazwa
Jan	Kowalski	Laptop
Jan	Kowalski	Telefon
Anna	Nowak	Rower
Piotr	Zieliński	Książka
Kasia	Wiśniewska	Plecak
Patryk	Nowakowski	<i>NULL</i>

RIGHT JOIN - czyli wszystkie rekordy z prawej tabeli. W naszym przypadku prawa tabela to Przedmioty. Jeśli Przedmiot nie ma dopasowania w tabeli Osoby również się wyświetli.

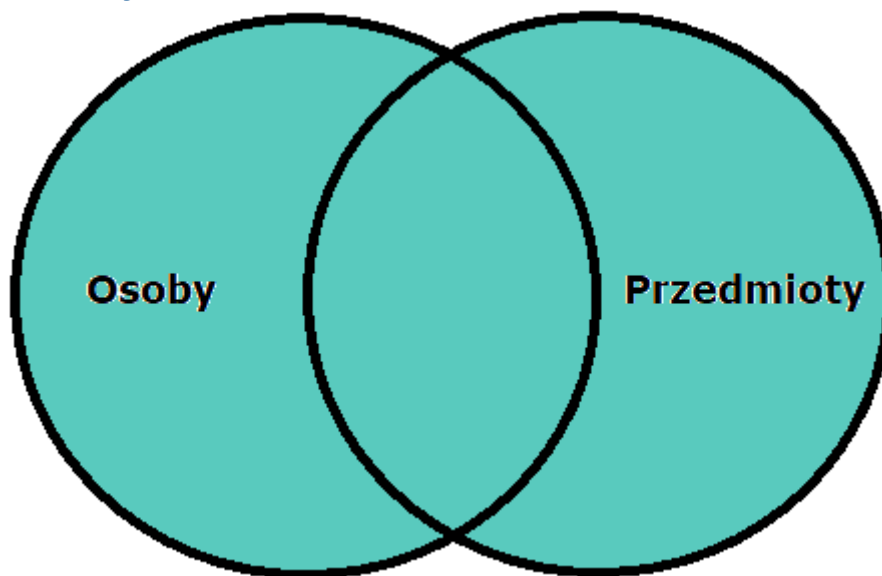


```
SELECT Osoby.imie, Osoby.nazwisko, Przedmioty.nazwa  
FROM Osoby  
RIGHT JOIN Przedmioty ON Osoby.osoba_id = Przedmioty.osoba_id;
```

Wynik:

imie	nazwisko	nazwa
Jan	Kowalski	Laptop
Jan	Kowalski	Telefon
Anna	Nowak	Rower
Piotr	Zieliński	Książka
Kasia	Wiśniewska	Plecak
<i>NULL</i>	<i>NULL</i>	Kubek

FULL OUTER JOIN (LEFT JOIN, UNION, RIGHT JOIN) - czyli wszystkie rekordy z prawej i lewej tabeli połączone. **W MySQL nie ma instrukcji FULL OUTER JOIN.** Jednak można wykonać ten mechanizm za pomocą połączenia poleceń right join, left join i UNION.



```
SELECT o.imie, o.nazwisko, p.nazwa  
FROM Osoby o  
LEFT JOIN Przedmioty p ON o.osoba_id = p.osoba_id
```

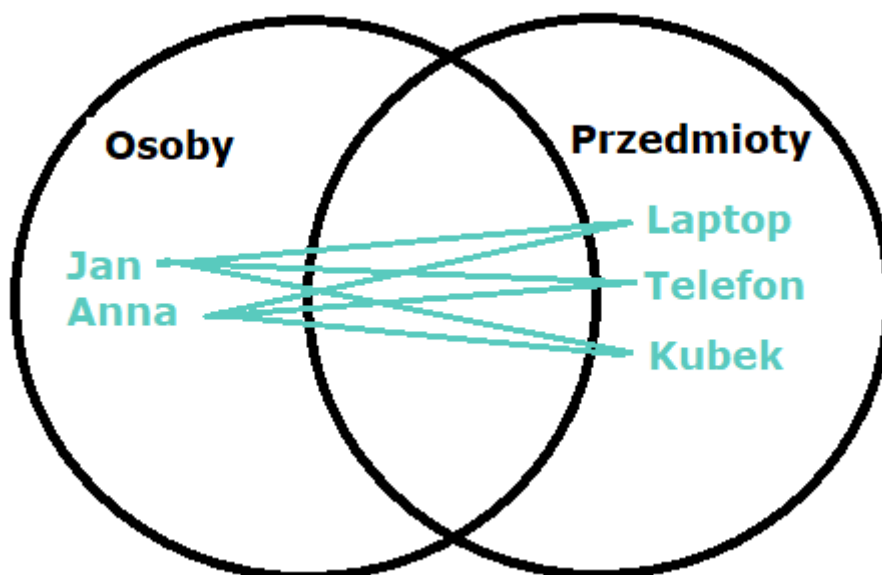
UNION

```
SELECT o.imie, o.nazwisko, p.nazwa  
FROM Osoby o  
RIGHT JOIN Przedmioty p ON o.osoba_id = p.osoba_id;
```

Wynik:

imie	nazwisko	nazwa
Jan	Kowalski	Laptop
Jan	Kowalski	Telefon
Anna	Nowak	Rower
Piotr	Zieliński	Książka
Kasia	Wiśniewska	Plecak
Patryk	Nowakowski	NULL
NULL	NULL	Kubek

CROSS JOIN - łączy **każdy wiersz z pierwszej tabeli** z **każdym wierszem z drugiej tabeli**.



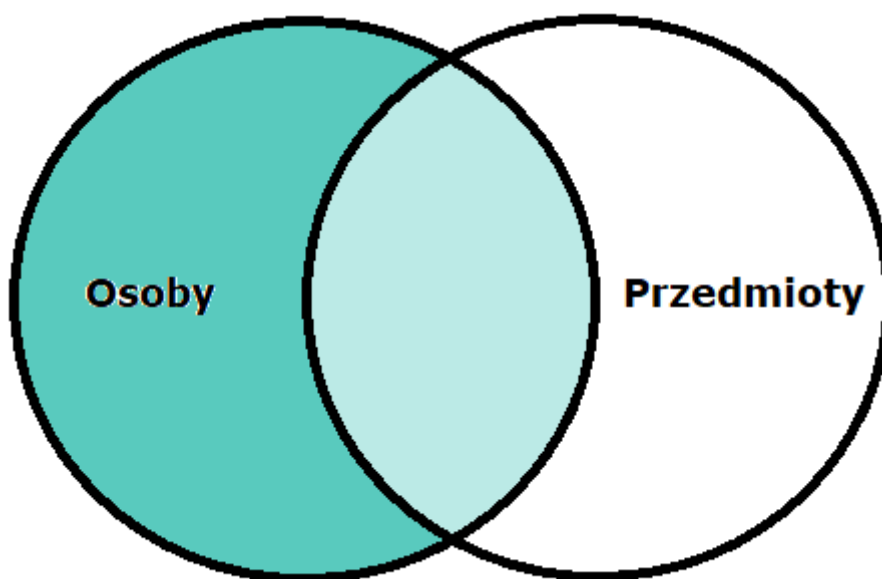
```
SELECT o.imie, p.nazwa  
FROM Osoby o  
CROSS JOIN Przedmioty p;
```

Wynik:

imie	nazwa
Jan	Laptop
Anna	Laptop
Piotr	Laptop
Kasia	Laptop
Patryk	Laptop
Jan	Telefon
Anna	Telefon
Piotr	Telefon
Kasia	Telefon
Patryk	Telefon
Jan	Rower
Anna	Rower
Piotr	Rower
Kasia	Rower
Patryk	Rower
Jan	Książka
Anna	Książka
Piotr	Książka
Kasia	Książka
Patryk	Książka
Jan	Plecak
Anna	Plecak
Piotr	Plecak
Kasia	Plecak
Patryk	Plecak
Jan	Kubek

Anna	Kubek
Piotr	Kubek
Kasia	Kubek
Patryk	Kubek

LEFT JOIN excluding INNER JOIN (LEFT JOIN wykluczający wiersze dopasowane) - **na początku wykonuje zapytanie LEFT JOIN. Następnie filtruje wynik wyświetlając z lewej tabeli wartości nie mających dopasowania w tabeli prawej.** Czyli w naszym przypadku z tabeli Osoby wyświetli wartości, które nie mają dopasowania

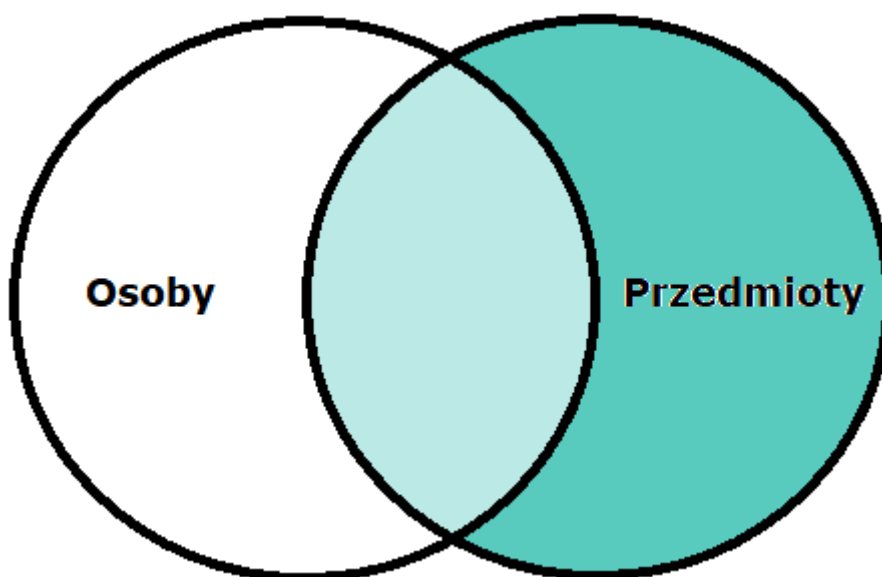


```
SELECT o.imie, o.nazwisko, p.nazwa
FROM Osoby o
LEFT JOIN Przedmioty p ON o.osoba_id = p.osoba_id
WHERE p.osoba_id IS NULL;
```

Wynik:

imie	nazwisko	nazwa
Patryk	Nowakowski	<i>NULL</i>

RIGHT JOIN excluding INNER JOIN (RIGHT JOIN wykluczający wiersze dopasowane) - **na początku wykonuje zapytanie RIGHT JOIN. Następnie filtruje wynik wyświetlając z prawej tabeli wartości nie mających dopasowania w tabeli lewej.** Czyli w naszym przypadku z tabeli Przedmioty wyświetli wartości, które nie mają dopasowania

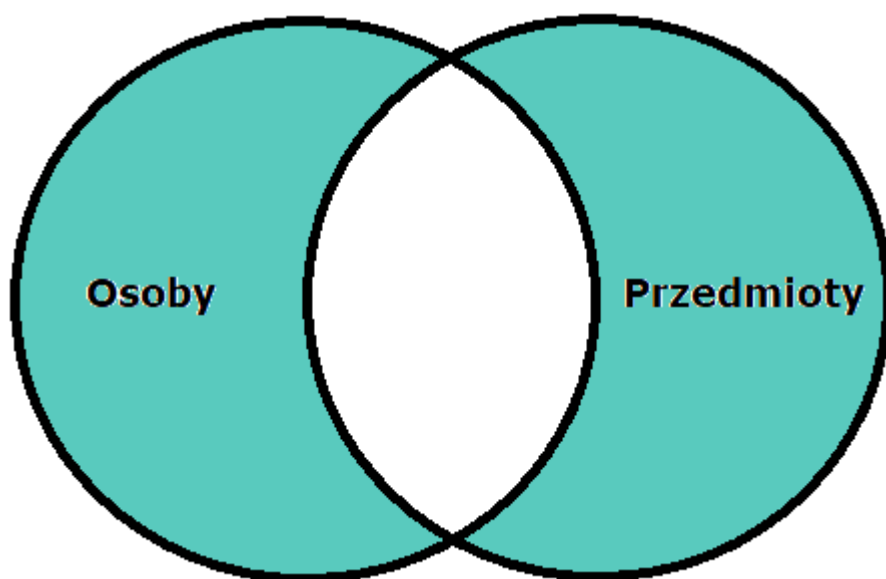


```
SELECT o.imie, o.nazwisko, p.nazwa
FROM Osoby o
RIGHT JOIN Przedmioty p ON o.osoba_id = p.osoba_id
WHERE o.osoba_id IS NULL;
```

Wynik:

imie	nazwisko	nazwa
<i>NULL</i>	<i>NULL</i>	Kubek

FULL OUTER JOIN excluding INNER JOIN (LEFT JOIN wykluczający wiersze dopasowane, UNION, RIGHT JOIN wykluczający wiersze dopasowane) - czyli wszystkie rekordy z prawej i lewej tabeli połączone. Następnie odrzucamy te wiersze, które mają dopasowanie w obu tabelach. **W MySQL nie ma instrukcji FULL OUTER JOIN.** Dla MySQL należy zastosować UNION. Czyli left join z wartościami nie mających dopasowania oraz right join z wartościami nie mających dopasowania łączymy z UNION.



```
SELECT o.imie, o.nazwisko, p.nazwa
FROM Osoby o
```

LEFT JOIN Przedmioty p ON o.osoba_id = p.osoba_id
WHERE p.osoba_id IS NULL

UNION

SELECT o.imie, o.nazwisko, p.nazwa
FROM Osoby o
RIGHT JOIN Przedmioty p ON o.osoba_id = p.osoba_id
WHERE o.osoba_id IS NULL;

Wynik:

imie	nazwisko	nazwa
Patryk	Nowakowski	NULL
NULL	NULL	Kubek

Lekcja 3

Temat: SQL – strukturalny język zapytań. Standardy, składnia języka SQL. Terminatory. Hierarchia obiektów bazy danych

SQL (ang. Structured Query Language) to standaryzowany, deklaratywny język programowania używany do zarządzania, manipulowania i pobierania danych z relacyjnych baz danych.

Standardy języka SQL

W celu ujednolicenia wersji języka SQL **Amerykański Narodowy Instytut Standardów** (ang. **American National Standards Institute — ANSI**) i **Międzynarodowa Organizacja Normalizacyjna** (ang. **International Organization for Standardization — ISO**) opracowują i publikują standardy języka SQL.

Standardy te ewoluowały od lat 80., dodając nowe funkcje, takie jak wsparcie dla JSON, zapytań grafowych czy zaawansowanego przetwarzania danych. Poniżej chronologiczna lista głównych wersji standardu SQL (na podstawie ANSI/ISO):

- **SQL-86** (ANSI X3.135:1986) – Pierwszy standard, podstawowe operacje na bazach relacyjnych.
- **SQL-87** (ISO/IEC 9075:1987) – Adopcja przez ISO, niewielkie poprawki.
- **SQL-89** (ANSI X3.135:1989, ISO/IEC 9075:1989) – Poprawki błędów i ujednolicenie.
- **SQL-92** (ANSI X3.135-1992, ISO/IEC 9075:1992) – Rozszerzony standard, dodano podzapytania, triggerzy i schematy.
- **SQL:1999** (ISO/IEC 9075:1999) - Wprowadzono typy obiektowe, rekurencyjne zapytania i procedury składowane.
- **SQL:2003** (ISO/IEC 9075:2003) – Dodano wsparcie dla XML i okienkowych funkcji.
- **SQL:2006** (ISO/IEC 9075:2006) – Rozszerzenia dla XML i zarządzania danymi.
- **SQL:2008** (ISO/IEC 9075:2008) – Ulepszenia w okienkowych funkcjach i triggerzy.
- **SQL:2011** (ISO/IEC 9075:2011) – Dodano temporalne tabele (historia danych) i sekwencje.
- **SQL:2016** (ISO/IEC 9075:2016) - Wprowadzono natywne wsparcie dla JSON i polimorficzne funkcje.
- **SQL:2023** (ISO/IEC 9075:2023) – Najnowsza wersja (stan na 2026), podzielona na 9 części, z nowościami jak **typ danych JSON**, **zapytania grafowe** (Property Graph Queries) i ulepszenia w wielowymiarowych tablicach.

Dialekt SQL to odmiana języka SQL używana przez konkretny system bazy danych. SQL jest standardem, ale każda baza danych dodaje własne funkcje, składnię i rozszerzenia. To właśnie nazywamy **dialektem SQL**.

Najbardziej znane to:

- **T-SQL (ang. Transact-SQL)** — stosowany na serwerach Microsoft SQL Server i Sybase Adaptive Server Enterprise,
- **PL/SQL (ang. Procedural Language/SQL)** — stosowany na serwerach firmy Oracle,
- **PL/pgSQL (ang. Procedural Language/PostgreSQL Structured Query Language)** — podobna do PL/SQL wersja języka zaimplementowana na serwerze PostgreSQL,
- **SQL PL (ang. SQL Procedural Language)** — wersja używana na serwerach firmy IBM.

Terminatory SQL (ang. SQL terminators) to specjalne znaki lub sekwencje znaków, które wskazują na koniec instrukcji SQL. Służą do oddzielania jednej komendy od drugiej, informując silnik bazy danych, że dana instrukcja jest kompletna i gotowa do wykonania. Najczęściej używanym terminatorem jest średnik (;), ale w zależności od systemu bazodanowego lub narzędzia (np. Interaktywnego klienta SQL) może być inny.

```
SELECT * FROM users; -- Koniec instrukcji
INSERT INTO users (name) VALUES ('Jan'); -- Kolejna instrukcja
```

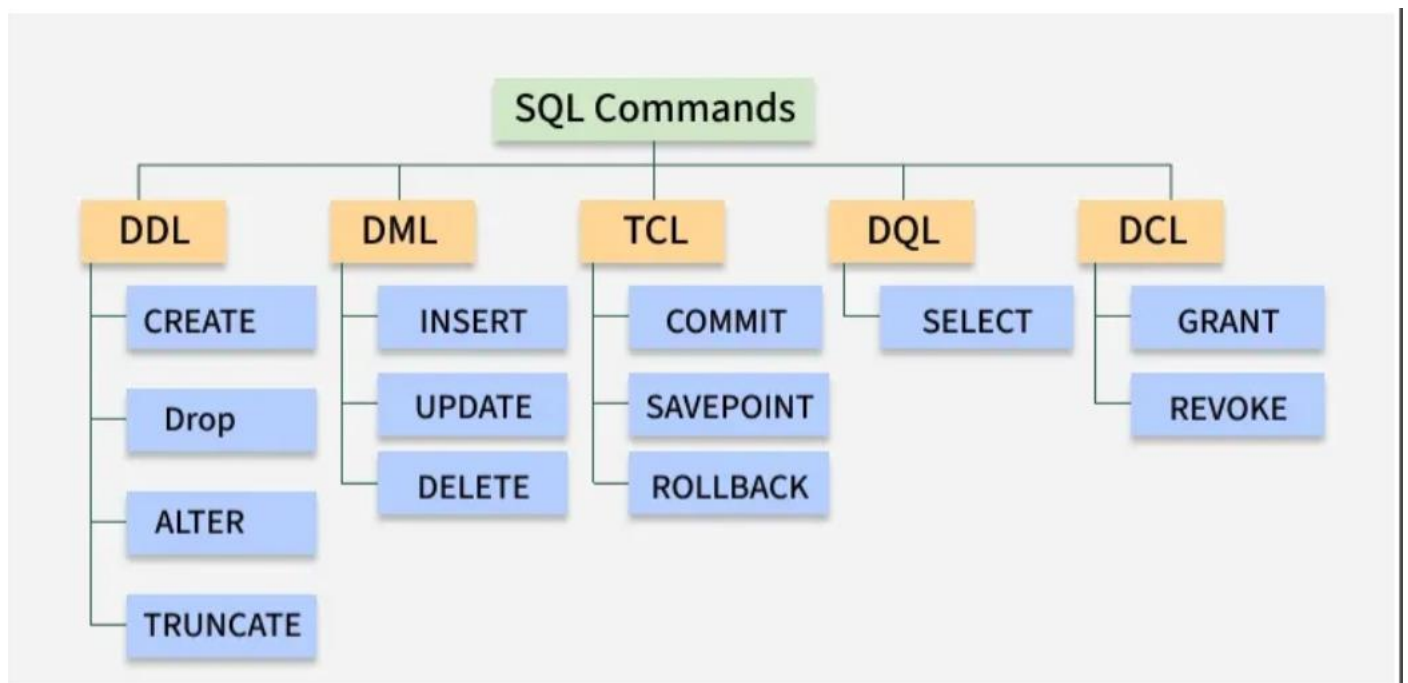
Terminatory wsadowe (ang. batch terminators) to specjalne komendy lub znaki używane w narzędziach do zarządzania bazami danych, które oddzielają partie (batches) kodu SQL. Batch to grupa instrukcji SQL, które są wysyłane do serwera jako jedna jednostka do wykonania. W przeciwieństwie do terminatorów instrukcji (statement terminators, np. ;), które kończą pojedyncze polecenia, **terminatory wsadowe dzielą skrypt na niezależne sekcje**

```
CREATE TABLE users (id INT);
GO -- Terminator wsadowy, kończy batch
INSERT INTO users (id) VALUES (1);
GO
```

Składnia języka SQL

Klasyfikację poleceń ze względu na ich przeznaczenie. Są to:

- **Instrukcje DDL (ang. Data Definition Language)** —tworzą język definiowania danych i służą do tworzenia, modyfikowania i usuwania obiektów bazy danych.
- **Instrukcje DML (ang. Data Manipulation Language)** —tworzą język manipulowania danymi i służą do odczytywania i modyfikowania danych.
- **Instrukcje DCL (ang. Data Control Language)** —tworzą język kontroli dostępu do danych i umożliwiają nadawanie i odbieranie uprawnień użytkownikom.
- **Instrukcje TCL (ang. Transaction Control Language)** —tworzą język obsługi transakcji.
- **Instrukcja DQL (ang. Data Querying Language)** — tworzy język definiowania zapytań, który zawiera tylko jedno polecenie SELECT i służy do tworzenia kwerend (zapytań).



Hierarchia obiektów bazy danych

Obiekty dostępne na serwerze bazodanowym tworzą hierarchię:

zawiera wiele baz danych, baza danych może zawierać wiele schematów, w każdym schemacie może występować wiele tabel, a każda tabela może składać się z wielu kolumn.

Każdy z tych obiektów powinien mieć nadaną niepowtarzalną nazwę (ang. alias). Przy odwoływaniu się w instrukcjach do obiektu jego nazwa powinna zawierać następujące elementy:

nazwa_serwera.nazwa_bazy_danych.nazwa_schematu.nazwa_obiektu

- Dla MySQL:

Server

└─ Database (= Schema)

└─ Table

└─ Column

W praktyce niektóre z tych elementów mogą zostać pominięte.

- **Nazwa serwera wskazuje serwer bazodanowy, na którym znajduje się obiekt.** Jeżeli ten element zostanie pominięty, to instrukcja zostanie wykonana przez serwer, z którym jesteśmy połączeni.
- **Nazwa bazy danych wskazuje bazę danych, w której znajduje się obiekt.** Jeżeli pominiemy tę nazwę, serwer wykona instrukcję w bazie danych, z którą jesteśmy połączeni.
- **Nazwa schematu wskazuje schemat, w którym znajduje się obiekt. Schemat jest zbiorem powiązanych ze sobą obiektów, tworzonym w bazie danych przez użytkownika w celu usprawnienia administrowania bazami danych.** Jeżeli nazwa schematu zostanie pominięta, serwer przyjmie, że obiekt znajduje się w domyślnym schemacie użytkownika wykonującego instrukcję. Jeżeli nie zadeklarujemy schematu, domyślnym schematem użytkownika staje się schemat **dbo** (dla MS SQL).
- **Różni użytkownicy bazy danych mogą mieć przypisane różne schemat domyślne, dlatego podając nazwę schematu, unikniemy ewentualnych błędów** Poza tym posługiwanie się nazwami schematów skraca czas wykonania instrukcji.

Lekcja 7

Temat: Instrukcje DDL (ang. Data Definition Language) w MySQL. Create database.

Link do dokumentacji technicznej MySQL odnośnie tworzenia bazy danych:
[dokumentacji MySQL - create database](#)

CREATE DATABASE

`CREATE DATABASE` tworzy bazę danych o podanej nazwie. Aby użyć tego polecenia, potrzebujesz `CREATE` uprawnienia do bazy danych. `CREATE SCHEMA` jest synonimem `CREATE DATABASE`. Błąd wystąpi, jeśli baza danych istnieje i nie określono jej `IF NOT EXISTS`

W sesji zawierającej aktywne blokady założone poleceniem `LOCK TABLES` wykonanie polecenia `CREATE DATABASE` nie jest możliwe.

Zablokowanie tabeli poleceniem `LOCK TABLES klienci READ`; lub `LOCK TABLES klienci WRITE`; uniemożliwia tworzenie bazy danych

Polecenie `UNLOCK TABLES`; (tu nie podajemy nazwy tabeli, zwalnia **wszystkie blokady** nałożone w sesji) powoduje możliwość dodawania baz danych.

Przykład:

```
CREATE TABLE klienci (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    nazwa VARCHAR(255)  
);  
CREATE DATABASE test1;  
LOCK TABLES klienci WRITE;  
CREATE DATABASE test2;  
UNLOCK TABLES;  
CREATE DATABASE test3;
```

```
CREATE {DATABASE | SCHEMA} [IF NOT EXISTS] db_name  
    [create_option] ...
```

```
create_option: [DEFAULT] {  
    CHARACTER SET [=] charset_name  
    | COLLATE [=] collation_name  
    | ENCRYPTION [=] {'Y' | 'N'}  
}
```

Opcja DEFAULT CHARACTER SET (lub po prostu CHARACTER SET)

- Ustawia domyślny zestaw znaków dla tabel w bazie (np. utf8mb4 dla pełnego wsparcia Unicode, latin1 dla starszych systemów).
- Dostępne zestawy: utf8mb4, utf8, latin1, cp1250 (dla polskiego), itp. Sprawdź za pomocą `SHOW CHARACTER SET`

`CREATE DATABASE moja_baza DEFAULT CHARACTER SET utf8mb4; # polskie znaki`

- o Z równością (opcjonalne):

`CREATE DATABASE moja_baza CHARACTER SET = utf8mb4;`

Opcja DEFAULT COLLATE (lub po prostu COLLATE)

- Ustawia domyślne sortowanie (collation) dla porównań ciągów znaków (np. case-insensitive, akcenty) m.in.:
 - o czy wielkość liter ma znaczenie (A = a czy nie)
 - o jak sortowane są polskie znaki (ą, ę, ł)
 - o jak działa porównywanie tekstów (=, LIKE, ORDER BY)
- Musi być zgodne z CHARACTER SET (np. dla utf8mb4: utf8mb4_general_ci, utf8mb4_0900_ai_ci).

- Sprawdź dostępne: `SHOW COLLATION;`

`CREATE DATABASE moja_baza DEFAULT COLLATE utf8mb4_general_ci;`

- o Z równością

`CREATE DATABASE moja_baza COLLATE = utf8mb4_general_ci;`

- o Łączenie z CHARACTER SET

`CREATE DATABASE moja_baza
CHARACTER SET utf8mb4
COLLATE utf8mb4_general_ci;`

Ustawienie COLLATE dla takich wymagań

- **utf8mb4_bin**
 - o Porównuje znaki **dokładnie binarnie** – wszystkie różnice wielkości liter i diakrytyków są zachowane.
 - o Minus: kolejność liter nie będzie naturalna dla alfabetu polskiego (A, Ą, B, C), bo sortuje według wartości binarnych.
- **utf8mb4_0900_as_cs** (MySQL 8.0+)
 - o **as = accent sensitive** → rozróżnia znaki diakrytyczne (ą != a)
 - o **cs = case sensitive** → rozróżnia wielkie i małe litery (A != a)
 - o Kolejność liter w tabeli jest zgodna z Unicode, więc polskie znaki (ą, ć, ę, ł, ń, ó, ś, ż, ź) są sortowane prawidłowo w porównaniu do alfabetu.

Przykład tworzenia bazy zgodnie z tymi wymaganiami:

`CREATE DATABASE moja_baza
DEFAULT CHARACTER SET utf8mb4
DEFAULT COLLATE utf8mb4_unicode_520_ci;`

[2] Przykład w tabeli <https://onecompiler.com/mysql/44fyrwnza>

`CREATE TABLE klienci (
id INT PRIMARY KEY,`

```

    imie VARCHAR(50)
) DEFAULT CHARSET=utf8mb4
COLLATE utf8mb4_unicode_520_ci;

INSERT INTO klienci (id, imie) VALUES
(1, 'Ala'),
(2, 'ala'),
(3, 'Ąła'),
(4, 'ąła');

-- Sortowanie
SELECT imie FROM klienci ORDER BY imie;

```

Wynik:

1. Ala
2. ala
3. Ąła
4. ąła

Opcja DEFAULT ENCRYPTION (dostępna od MySQL 8.0)

- **Włącza lub wyłącza domyślne szyfrowanie tabel** (data-at-rest encryption) za pomocą klucza serwera.
- 'Y' – szyfrowane; 'N' – nieszyfrowane (domyślne to 'N', chyba że serwer ma włączone globalnie).
- Wymaga włączonego szyfrowania na serwerze (np. plugin keyring).

```
CREATE DATABASE moja_baza DEFAULT ENCRYPTION 'Y';
```

- o Z równością:

```
CREATE DATABASE moja_baza ENCRYPTION = 'N';
```

- o Pełny przykład z innymi opcjami:

```

CREATE DATABASE IF NOT EXISTS moja_baza
CHARACTER SET utf8mb4
COLLATE utf8mb4_general_ci
ENCRYPTION 'Y';

```

Ograniczenia: Nie możesz tworzyć bazy o nazwie zarezerwowanej (np. 'mysql') lub z niedozwolonymi znakami (używaj liter, cyfr, '_'; unikaj spacji).

Sprawdzanie po stworzeniu: Użyj SHOW CREATE DATABASE moja_baza; aby zobaczyć ustawienia, lub SHOW DATABASES; aby listę baz.

Link do dokumentacji technicznej MySQL odnośnie edytowania bazy danych:

ALTER DATABASE

Instrukcja `ALTER DATABASE` (lub synonim `ALTER SCHEMA`) służy do modyfikacji ogólnych właściwości istniejącej bazy danych w MySQL. Zmiany te są przechowywane w słowniku danych (data dictionary). Instrukcja wymaga uprawnień `ALTER` na bazie danych.

```
ALTER {DATABASE | SCHEMA} [nazwa_bazy]
    opcja_alter ...
```

```
opcja_alter: {
    [DEFAULT] CHARACTER SET [=] nazwa_zestawu_znakow
  | [DEFAULT] COLLATE [=] nazwa_sortowania
  | [DEFAULT] ENCRYPTION [=] {'Y' | 'N'}
  | READ ONLY [=] {DEFAULT | 0 | 1}
}
```

[DEFAULT] CHARACTER SET [=] nazwa_zestawu_znakow

Zmienia domyślny zestaw znaków (character set) dla bazy danych. Wpływa na domyślne zestawy dla nowych tabel i procedur składowanych. Po zmianie, istniejące procedury używające domyślnych wartości muszą być usunięte i odtworzone.

Dostępne zestawy sprawdzisz za pomocą `SHOW CHARACTER SET`; Popularne: `utf8mb4` (dla Unicode), `latin1`. Nie wpływa na istniejące tabele.

```
ALTER DATABASE moja_baza DEFAULT COLLATE utf8mb4_0900_ai_ci;
```

Przykład gdzie kolumna ma ustawione polskie znaki bez cerlicy. Czyli ukraińskie znaki nie zostaną zapisane poprawnie

```
CREATE DATABASE polska_baza
CHARACTER SET latin2
COLLATE latin2_general_ci; -- COLLATE dla ogólnego sortowania (nie wpływa na przechowywanie znaków)
```

```
CREATE TABLE test_znaki (
    id INT AUTO_INCREMENT PRIMARY KEY,
    tekst VARCHAR(50) CHARACTER SET latin2
);
```

```
-- Dodawanie polskiego znaku - działa
INSERT INTO test_znaki (tekst) VALUES ('Polski: ą ć ę ł ś');

-- Dodawanie ukraińskiego znaku - błąd lub uszkodzenie
INSERT INTO test_znaki (tekst) VALUES ('Ukraiński: і ї є'); -- Spodziewany Warning:
"Incorrect string value: '\xD1\x96 \xD2\x91 \xD1\x94' for column 'tekst'..."

-- Sprawdzenie zapisanych danych
SELECT * FROM test_znaki;
```

[DEFAULT] COLLATE [=] nazwa_sortowania

Zmienia domyślne sortowanie (collation) dla porównań ciągów znaków w bazie danych (np. uwzględnianie wielkości liter, akcentów).

Sortowanie musi być zgodne z bieżącym zestawem znaków. Dostępne opcje: SHOW COLLATION;. Przykłady: utf8mb4_general_ci (case-insensitive), utf8mb4_bin (binary). Zmiana może być powiązana z CHARACTER SET.

Przykład gdzie kolumna ma ustawione polskie znaki i jak one są do siebie identyczne

```
CREATE TABLE test_polski (
    id INT AUTO_INCREMENT PRIMARY KEY,
    litera VARCHAR(10) COLLATE utf8mb4_polish_ci
);

INSERT INTO test_polski (litera) VALUES ('a'), ('ą'), ('A'), ('Ą');

-- Porównanie: 'a' == 'ą' powinno być false (0)
SELECT 'a' = 'ą' AS porownanie1; -- Wynik: 0

-- Porównanie z wielką literą (case-insensitive, ale diakrytyki rozróżniane)
SELECT 'a' = 'A' AS porownanie2; -- Wynik: 1 (true, bo _ci ignoruje wielkość liter)
SELECT 'a' = 'Ą' AS porownanie3; -- Wynik: 0 (false)

-- Sortowanie: Pokazuje polskie reguły (ą po a)
SELECT litera FROM test_polski ORDER BY litera;
-- Oczekiwany wynik: 'a', 'A', 'ą', 'Ą' (polskie sortowanie traktuje ą jako po a)
```

[DEFAULT] ENCRYPTION [=] {'Y' | 'N'}

Ustawia domyślne szyfrowanie (data-at-rest encryption) dla nowych tabel w bazie. 'Y' włącza szyfrowanie, 'N' wyłącza (domyślne).

Dotyczy tylko nowych tabel; istniejące pozostają bez zmian. Nie działa dla systemowych baz: mysql, information_schema, performance_schema. Wymaga uprawnień TABLE_ENCRYPTION_ADMIN jeśli table_encryption_privilege_check jest włączone i ustawienie różni się od globalnego default_table_encryption. Używa klucza serwera.

```
ALTER DATABASE moja_baza DEFAULT ENCRYPTION 'Y';
```

READ ONLY [=] {DEFAULT | 0 | 1}

Kontroluje, czy baza jest tylko do odczytu (read-only). 1 – tylko odczyt (blokuje modyfikacje), 0 – zapisywalna, DEFAULT – resetuje do domyślnego serwera (zazwyczaj zapisywalna).

Nie działa dla systemowych baz.

```
ALTER DATABASE moja_baza READ ONLY = 1;
```

Przykłady łączone

- Zmiana wielu opcji:

```
ALTER DATABASE moja_baza
```

```
CHARACTER SET utf8mb4
```

```
COLLATE utf8mb4_bin
```

```
ENCRYPTION 'N'
```

```
READ ONLY = 0;
```

- Sprawdzenie bieżących ustawień:

```
SHOW CREATE DATABASE moja_baza\G
```

Link do dokumentacji technicznej MySQL odnośnie usuwania bazy danych:
[dokumentacji MySQL - drop database](#)

DROP DATABASE

Instrukcja DROP DATABASE (lub synonim DROP SCHEMA) w MySQL to część DDL (Data Definition Language) i służy do **permanentnego usunięcia całej bazy danych**. Usuwa **nie tylko samą bazę, ale także wszystkie zawarte w niej obiekty: tabele, widoki, procedury, triggerzy, indeksy, użytkowników przypisanych do bazy (jeśli specyficzni), a także fizyczne pliki na dysku**. Jest to operacja nieodwracalna – **nie można jej cofnąć za pomocą ROLLBACK** (DDL w MySQL automatycznie commituje transakcje).

```
DROP {DATABASE | SCHEMA} [IF EXISTS] nazwa_bazy;
```

Gdy wykonujesz DROP DATABASE [IF EXISTS] nazwa_bazy;, MySQL przetwarza to w następujących etapach (uproszczone, dla silników jak InnoDB/MyISAM; szczegóły zależą od konfiguracji serwera):

1. Sprawdzenie uprawnień i istnienia:

Weryfikuje uprawnienia DROP (np. root). Sprawdza, czy baza istnieje (błąd jeśli nie, chyba że IF EXISTS). Jeśli baza w użyciu (połączenia, locki), może czekać lub zgłosić błąd (nie blokuje automatycznie, co jest ryzykowne).

2. Obsługa powiązań i zależności:

Rekurencyjnie usuwa wszystkie obiekty wewnątrz: tabele (z danymi, indeksami, foreign keys – ignoruje RESTRICT), widoki, procedury, triggerzy, events. Usuwa przywileje użytkowników

specyficzne dla bazy (ale nie samych użytkowników). Nie wpływa na globalne ustawienia serwera. W replikacji propaguje zmianę do slave'ów (może spowodować błędy).

3. **Usuwanie metadanych i plików fizycznych:**

Kasuje wpis bazy z data dictionary (information_schema, mysql). Fizycznie usuwa katalog bazy na dysku (np. /var/lib/mysql/nazwa_bazy/): pliki .ibd (InnoDB), .MYD/.MYI/.frm (MyISAM). Resetuje cache. Obsługuje szyfrowanie jeśli włączone.

4. **Commit i logowanie:**

Automatycznie commituje (implicit commit DDL). Loguje w binary log dla replikacji/recovery (odtworzalne, ale nie cofalne bez backupu). W klastrach synchronizuje przez węzły.

5. **Konsekwencje po usunięciu:**

Baza znika (SHOW DATABASES; jej nie widzi). Przerywa połączenia (błędy przy USE). Dane utracone na zawsze (chyba że backup/logi). Dla dużych baz – może blokować serwer i trwać długo. Sesja traci kontekst jeśli baza była domyślna.

Lekcja **ZAPOWIEDZIEĆ SPRAWDZIAN i weryfikacja projektów**

Temat: TRUNCATE i DROP.

Link do dokumentacji MySQL do instrukcji TRUNCATE

<https://dev.mysql.com/doc/refman/9.6/en/truncate-table.html>

Instrukcja TRUNCATE TABLE w MySQL należy do kategorii DDL (Data Definition Language), choć czasem klasyfikowana jest na granicy z DML (Data Manipulation Language), ponieważ **modyfikuje strukturę danych poprzez usunięcie wszystkich wierszy z tabeli**.

Jest to szybki sposób na opróżnienie tabeli bez usuwania jej samej. W przeciwieństwie do DELETE FROM tabela (bez klauzuli WHERE), TRUNCATE jest efektywniejsze, ponieważ nie loguje każdej usuniętej operacji indywidualnie (co czyni je szybszym dla dużych tabel), **resetuje liczniki AUTO_INCREMENT do 1 i nie wyzwala triggerów BEFORE/AFTER DELETE (jeśli istnieją)**.

Proces: MySQL symuluje usunięcie tabeli i jej ponowne utworzenie z tą samą strukturą, ale bez danych. To sprawia, że operacja jest atomiczna i szybsza.

Ograniczenia:

- **Nie zwraca liczby usuniętych wierszy** (w przeciwieństwie do DELETE).
- **Wymaga uprawnień DROP na tabelę.**
- Operacji truncate nie można wykonać, jeśli sesja trzyma aktywny lock na tabeli.

Kiedy używać?: Do czyszczenia dużych zbiorów danych testowych, resetowania tabel w aplikacjach, lub gdy nie potrzebujesz logowania zmian (np. w hurtowniach danych).

Różnice z DELETE: DELETE pozwala na WHERE (usuwanie selektywne), wyzwała triggery, loguje zmiany (wolniejsze), nie resetuje AUTO_INCREMENT.

1. Przykłady zastosowań

```
CREATE TABLE uzytkownicy (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    nazwa VARCHAR(255)  
);
```

```
INSERT INTO uzytkownicy (nazwa) VALUES ('Jan'), ('Anna');  
INSERT INTO uzytkownicy (nazwa) VALUES ('Jan'), ('Anna');  
Select * from uzytkownicy;
```

```
TRUNCATE TABLE uzytkownicy;
```

```
Select * from uzytkownicy;  
INSERT INTO uzytkownicy (nazwa) VALUES ('Jan'), ('Anna');  
INSERT INTO uzytkownicy (nazwa) VALUES ('Jan'), ('Anna');  
Select * from uzytkownicy;
```

-- Wynik: Tabela jest pusta, AUTO_INCREMENT zaczyna od 1. Żadne dane nie są zwracane.

2. Obsługa foreign keys (wyłączanie sprawdzania):

Jeśli tabela ma powiązania, TRUNCATE zgłosi błąd. Usunięcie zawartości za pomocą TRUNCATE, można wykonać za pomocą usunięcia klucza obcego, poczym go ponownie dodanie.

```
CREATE TABLE klienci (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    nazwa VARCHAR(255)  
);  
  
CREATE TABLE zamowienia (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    klient_id INT,  
    CONSTRAINT fk_zamowienia_klient  
    FOREIGN KEY (klient_id) REFERENCES klienci(id)  
);
```

```
INSERT INTO klienci (nazwa) VALUES ('Firma A');  
INSERT INTO zamowienia (klient_id) VALUES (1);  
SELECT  
k.id AS klient_id,  
k.nazwa AS klient_nazwa,  
z.id AS zamowienie_id  
FROM klienci k  
JOIN zamowienia z ON k.id = z.klient_id;
```

```
ALTER TABLE zamowienia DROP FOREIGN KEY fk_zamowienia_klient;  
TRUNCATE TABLE zamowienia;
```

```
TRUNCATE TABLE klienci;  
ALTER TABLE zamowienia ADD CONSTRAINT fk_zamowienia_klient  
FOREIGN KEY (klient_id) REFERENCES klienci(id);  
  
SELECT  
k.id AS klient_id,  
k.nazwa AS klient_nazwa,  
z.id AS zamowienie_id  
FROM klienci k  
JOIN zamowienia z ON k.id = z.klient_id;
```

Podczas operacji TRUNCATE TABLE, MySQL usuwa te pliki i tworzy nowe, puste – co skutkuje zwolnieniem (oddaniem) zajętego miejsca z powrotem do systemu plików, zmniejszając tym samym zużycie dysku.

DROP

Instrukcja DROP w MySQL to polecenie DDL (Data Definition Language), które **służy do usuwania obiektów bazy danych, takich jak tabele, bazy danych, widoki, indeksy czy procedury**. DROP TABLE całkowicie usuwa tabelę wraz z jej definicją, danymi, indeksami i powiązаныmi triggerami. Operacja jest nieodwracalna bez backupu, więc używaj jej ostrożnie.

Proces: MySQL usuwa pliki tabeli (dane, indeksy) i aktualizuje metadane bazy. Dla tabel partycjonowanych usuwa wszystkie partycje i ich definicje.

Wymagania: Wymaga uprawnień DROP dla danej tabeli. Powoduje implicit commit (automatyczne zatwierdzenie transakcji), chyba że użyjesz opcji TEMPORARY.

Opcje kluczowe:

- **IF EXISTS:** Zapobiega błędom, jeśli tabela nie istnieje – ignoruje nieistniejące tabele i generuje notatki (sprawdź SHOW WARNINGS).
- **TEMPORARY:** Usuwa tylko tymczasowe tabele z bieżącej sesji. Nie sprawdza uprawnień i nie powoduje commitu.
- **RESTRICT / CASCADE:** W MySQL nie mają efektu (służą do kompatybilności z innymi DBMS), ale można je dodać.

Ograniczenia: Nie działa, jeśli tabela jest zablokowana lub pod pewnymi ustawieniami odzyskiwania (np. innodb_force_recovery). Uprawnienia nadane na tabeli nie są automatycznie usuwane – użyj REVOKE.

Różnice z innymi instrukcjami: W przeciwieństwie do TRUNCATE TABLE (które opróżnia tabelę, ale zachowuje strukturę), **DROP TABLE całkowicie usuwa obiekt. Jest podobne do DELETE, ale działa na poziomie struktury.**

Przykłady zastosowań

1. **Podstawowe usunięcie tabeli:** Usuwa tabelę pracownicy wraz z danymi.

```
DROP TABLE pracownicy;
```

Wynik: Jeśli tabela istnieje, zostaje usunięta. Jeśli nie – błąd.

Zastosowanie: Czyszczenie niepotrzebnych tabel w bazie deweloperskiej.

2. Usunięcie wielu tabel z opcją IF EXISTS: Usuwa tabele, jeśli istnieją, bez błędów.

```
DROP TABLE IF EXISTS tabela1, tabela2, tabela3;
```

Wynik: Nieistniejące tabele są ignorowane; użyj SHOW WARNINGS, aby zobaczyć notatki.

Zastosowanie: W skryptach migracyjnych, gdzie tabela może już nie istnieć.

3. Usunięcie tymczasowej tabeli: Usuwa tylko tymczasową tabelę z sesji.

```
CREATE TEMPORARY TABLE temp_dane (id INT);
```

```
DROP TEMPORARY TABLE temp_dane;
```

```
DROP TEMPORARY TABLE temp_dane;
```

Wynik: Tabela usunięta bez wpływu na stałe tabele o tej samej nazwie.

Zastosowanie: W sesyjnych obliczeniach, np. w raportach, aby uniknąć kolizji z stałymi tabelami.

4. Usunięcie z opcją CASCADE (dla kompatybilności):

```
DROP TABLE IF EXISTS klienci CASCADE;
```

Wynik: W MySQL CASCADE nie ma efektu (nie usuwa automatycznie powiązanych obiektów), ale polecenie działa.

Zastosowanie: W kodzie migrującym z innych baz danych, jak PostgreSQL, gdzie CASCADE usuwa zależności.

Lekcja

Temat: Sprawdzian wiadomości.

Lekcja

Temat: JOIN w SQL. Ćwiczenia praktyczne z zapytań

```
CREATE TABLE klienci (  
  klient_id INT PRIMARY KEY,  
  imie      VARCHAR(50),  
  miasto    VARCHAR(50)
```

```
);
```

```
INSERT INTO klienci VALUES
```

```
(1, 'Jan Kowalski', 'Warszawa'),  
(2, 'Anna Nowak', 'Kraków'),  
(3, 'Piotr Wiśniewski', 'Gdańsk'),  
(4, 'Maria Zielińska', 'Wrocław'),  
(5, 'Tomasz Lewandowski', 'Poznań');
```

```
CREATE TABLE zamowienia (
```

```
zamowienie_id INT PRIMARY KEY,  
klient_id INT,  
kwota DECIMAL(10,2),  
data_zamowienia DATE  
);
```

```
INSERT INTO zamowienia VALUES
```

```
(101, 1, 450.00, '2025-01-15'),  
(102, 2, 1200.50, '2025-02-10'),  
(103, 1, 320.00, '2025-02-20'),  
(104, 3, 890.00, '2025-03-05'),  
(105, 6, 150.00, '2025-03-10'); -- klient_id = 6 → nie istnieje!
```

INNER JOIN

Zwraca **tylko wiersze, które mają dopasowanie w obu tabelach.**

```
SELECT
```

```
k.imie,  
k.miasto,  
z.zamowienie_id,  
z.kwota,  
z.data_zamowienia
```

```
FROM klienci k
```

```
INNER JOIN zamowienia z ON k.klient_id = z.klient_id;
```

imie	miasto	zamowienie_id	kwota	data_zamowienia
Jan Kowalski	Warszawa	101	450.00	2025-01-15
Anna Nowak	Kraków	102	1200.50	2025-02-10
Jan Kowalski	Warszawa	103	320.00	2025-02-20
Piotr Wiśniewski	Gdańsk	104	890.00	2025-03-05

LEFT JOIN (LEFT OUTER JOIN)

Zwraca **wszystkich klientów** + ich zamówienia (jeśli nie ma zamówienia → NULL).

SELECT

k.imie,
k.miasto,
z.zamowienie_id,
z.kwota

FROM klienci k

LEFT JOIN zamówienia z ON k.klient_id = z.klient_id

ORDER BY k.klient_id;

imie	miasto	zamowienie_id	kwota
Jan Kowalski	Warszawa	101	450.00
Jan Kowalski	Warszawa	103	320.00
Anna Nowak	Kraków	102	1200.50
Piotr Wiśniewski	Gdańsk	104	890.00
Maria Zielińska	Wrocław	NULL	NULL
Tomasz Lewandowski	Poznań	NULL	NULL

RIGHT JOIN (RIGHT OUTER JOIN)

Zwraca **wszystkie zamówienia** + dane klienta (jeśli klienta nie ma → NULL).

```

SELECT
    k.imie,
    k.miasto,
    z.zamowienie_id,
    z.kwota
FROM klienci k
RIGHT JOIN zamowienia z ON k.klient_id = z.klient_id
ORDER BY z.zamowienie_id;

```

imie	miasto	zamowienie_id	kwota
Jan Kowalski	Warszawa	101	450.00
Anna Nowak	Kraków	102	1200.50
Jan Kowalski	Warszawa	103	320.00
Piotr Wiśniewski	Gdańsk	104	890.00
NULL	NULL	105	150.00

Złota zasada

- **Lewa tabela (left table)** = tabela, która stoi **bezpośrednio po słowie FROM**
- **Prawa tabela (right table)** = tabela, która stoi **bezpośrednio po słowie JOIN** (lub LEFT JOIN, RIGHT JOIN, INNER JOIN itp.)

Ćwiczenia praktyczne z zapytań

-- Tabela 1: Działy

```

CREATE TABLE IF NOT EXISTS dzialy (
    id      INT PRIMARY KEY AUTO_INCREMENT,
    nazwa   VARCHAR(50) NOT NULL UNIQUE
);

```

-- Tabela 2: Pracownicy

```

CREATE TABLE IF NOT EXISTS pracownicy (
    id      INT PRIMARY KEY AUTO_INCREMENT,
    imie    VARCHAR(50) NOT NULL,
    nazwisko VARCHAR(50) NOT NULL,
    dzial_id INT,

```

```

pensja    DECIMAL(10,2) DEFAULT 0.00,
data_zatrudnienia DATE DEFAULT (CURRENT_DATE),
FOREIGN KEY (dzial_id) REFERENCES dzialy(id)
    ON DELETE SET NULL
);

```

-- Tabela 3: Projekty

```

CREATE TABLE IF NOT EXISTS projekty (
    id          INT PRIMARY KEY AUTO_INCREMENT,
    nazwa_projektu VARCHAR(100) NOT NULL,
    pracownik_id INT,
    data_rozpoczecia DATE NOT NULL,
    status      ENUM('planowany', 'w realizacji', 'zakończony') DEFAULT 'planowany',
    FOREIGN KEY (pracownik_id) REFERENCES pracownicy(id)
        ON DELETE CASCADE
);

-- Wstawiamy działy (INSERT IGNORE – warunek: nie duplikujemy nazwy)
INSERT IGNORE INTO dzialy (nazwa) VALUES
    ('Sprzedaż'),
    ('IT'),
    ('HR'),
    ('Finanse');

```

-- Wstawiamy pracowników

```

INSERT INTO pracownicy (imie, nazwisko, dzial_id, pensja, data_zatrudnienia) VALUES
    ('Jan',      'Kowalski',  1, 5200.00, '2023-02-15'),
    ('Anna',     'Nowak',     2, 7800.00, '2024-01-10'),
    ('Piotr',    'Wiśniewski', 1, 4800.00, '2023-11-20'),
    ('Katarzyna', 'Zielińska', 3, 6500.00, '2024-03-05'),
    ('Marek',    'Lewandowski', 2, 9200.00, '2022-08-01'),
    ('Ola',      'Jankowska', 4, 6100.00, '2024-06-01');

```

```

INSERT INTO projekty (nazwa_projektu, pracownik_id, data_rozpoczecia, status)
SELECT * FROM (
    SELECT 'System CRM', id, '2024-09-01', 'w realizacji'
    FROM pracownicy WHERE imie = 'Anna' AND nazwisko = 'Nowak'
    UNION
    SELECT 'Strona www', id, '2024-10-15', 'planowany'
    FROM pracownicy WHERE imie = 'Marek' AND nazwisko = 'Lewandowski'
    UNION
    SELECT 'Szkolenie onboarding', id, '2025-01-10', 'planowany'
    FROM pracownicy WHERE imie = 'Katarzyna' AND nazwisko = 'Zielińska'
    UNION
    SELECT 'Kampania marketingowa', id, '2024-11-01', 'w realizacji'
    FROM pracownicy WHERE imie = 'Jan' AND nazwisko = 'Kowalski'
) AS tmp;

```

Zadania bez JOIN (4 zadania):

1. Wyświetl wszystkich pracowników (imię, nazwisko, pensja) posortowanych malejąco według pensji.

2. Dodaj nowego pracownika: Tomasz Kowalczyk, dział IT (id=2), pensja 8500 zł, data zatrudnienia dzisiejsza.
3. Zmień pensję wszystkim pracownikom z działu Sprzedaż (id=1) – podnieś o 10%. Użyj warunku.
4. Usuń projekt, który ma status „planowany” i nazwę zawierającą słowo „onboarding”.

Zadania z JOIN (4 zadania):

5. Wyświetl listę pracowników razem z nazwą działu (imie, nazwisko, nazwa działu, pensja).
6. Pokaż wszystkie projekty razem z imieniem i nazwiskiem pracownika oraz nazwą działu (użyj INNER JOIN).
7. Znajdź pracowników, którzy **nie** mają przypisanego żadnego projektu (LEFT JOIN + IS NULL).
8. Policz ile projektów jest w każdym dziale (GROUP BY + JOIN). Wyświetl nazwę działu i liczbę projektów.

-- 1.

```
SELECT imie, nazwisko, pensja
FROM pracownicy
ORDER BY pensja DESC;
```

-- 2.

```
INSERT INTO pracownicy (imie, nazwisko, dzial_id, pensja, data_zatrudnienia)
VALUES ('Tomasz', 'Kowalczyk', 2, 8500.00, CURDATE());
```

-- 3.

```
UPDATE pracownicy
SET pensja = pensja * 1.10
WHERE dzial_id = 1;
```

-- 4.

```
DELETE FROM projekty
WHERE status = 'planowany'
AND nazwa_projektu LIKE '%onboarding%';
```

-- 5.

```
SELECT p.imie, p.nazwisko, d.nazwa AS dzial, p.pensja
FROM pracownicy p
JOIN dzialy d ON p.dzial_id = d.id;
```

-- 6.

```
SELECT pr.nazwa_projektu, p.imie, p.nazwisko, d.nazwa AS dzial
FROM projekty pr
JOIN pracownicy p ON pr.pracownik_id = p.id
JOIN dzialy d ON p.dzial_id = d.id;
```

-- 7.

```
SELECT p.imie, p.nazwisko, d.nazwa AS dzial
```

```

FROM pracownicy p
LEFT JOIN projekty pr ON p.id = pr.pracownik_id
JOIN działy d ON p.dzial_id = d.id
WHERE pr.id IS NULL;

-- 8.
SELECT d.nazwa AS dzial, COUNT(pr.id) AS liczba_projektow
FROM działy d
LEFT JOIN pracownicy p ON d.id = p.dzial_id
LEFT JOIN projekty pr ON p.id = pr.pracownik_id
GROUP BY d.id, d.nazwa
ORDER BY liczba_projektow DESC;

```

Lekcja

Temat: Tabele tymczasowe w MySQL. Kompresja tabel i tymczasowych tabel.

Materiały do lekcji przygotowane na podstawie:

<https://dev.mysql.com/doc/refman/8.4/en/create-temporary-table.html>

Tabela TEMPORARY jest widoczna tylko w ramach bieżącej sesji i zostaje automatycznie usunięta po jej zamknięciu.

Dwie różne sesje (połączenia do bazy) mogą mieć tabelę tymczasową o tej samej nazwie (nazwa tabeli tymczasowej może być również identyczna jak tabela z której kopiujemy dane) i sobie nie przeszkadzają.

Przykład:

Sesja 1 – w jednej zakładce w przeglądarce

```

SELECT CONNECTION_ID();
CREATE TEMPORARY TABLE klienci AS
SELECT * FROM klienci;

```

```

SELECT * FROM klienci; // korzysta z TEMPORARY TABLE

```

Sesja 2 – w drugiej zakładce w przeglądarce

```

SELECT CONNECTION_ID();
SELECT * FROM klienci; // korzysta z normalnej tabeli (nie widzi tej tymczasowej!)

```

Polecenie sprawdzające zwracające unikalny identyfikator bieżącego połączenia (sesji) do MySQL

```

SELECT CONNECTION_ID();

```

Aby **utworzyć tabelę tymczasową**, musisz posiadać uprawnienie **CREATE TEMPORARY TABLES**. Po utworzeniu tabeli tymczasowej przez sesję serwer nie przeprowadza dalszych kontroli uprawnień dotyczących tej tabeli. **Sesja, która utworzyła tabelę, może wykonywać na niej dowolne operacje, takie jak DROP TABLE, INSERT, UPDATE lub SELECT.**

InnoDB nie obsługuje skompresowanych tabel tymczasowych. **Normalne tabele możesz kompresować**, ale **TEMPORARY TABLE nie może być skompresowana przy domyślnej opcji:**

`innodb_strict_mode = ON`

innodb_strict_mode to zmienna systemowa w MySQL, która kontroluje, jak silnik pamięci **InnoDB** reaguje na błędy lub nieprawidłową składnię podczas wykonywania poleceń takich jak `CREATE TABLE`, `ALTER TABLE` czy `CREATE INDEX`.

Różnica między trybem ścisłym a łagodnym

Tryb	Ustawienie	Zachowanie bazy danych
Ścisły (ON)	ON (lub 1)	Jeśli pojawi się błąd (np. nieprawidłowa opcja, zły typ danych), MySQL natychmiast przerywa działanie i wyrzuca błąd (ERROR) . Polecenie nie zostaje wykonane. Dzięki temu od razu wiesz, że coś jest nie tak.
Łagodny (OFF)	OFF (lub 0)	W tej samej sytuacji MySQL ignoruje problem, wyświetla tylko ostrzeżenie (WARNING) i próbuje wykonać polecenie, często zastępując błędną opcję domyślną wartością. Może to prowadzić do sytuacji, gdzie tabela tworzy się inaczej niż planowałeś, ale Ty nie widzisz żadnego czerwonego błędu.

Sprawdzenie aktualnego statusu dla trybu

```
SHOW VARIABLES LIKE 'innodb_strict_mode';
```

Zmiana tylko dla aktualnej sesji (tymczasowo)

```
SET SESSION innodb_strict_mode = OFF;
```

🔗 działa tylko w tej zakładce / połączeniu, po rozłączeniu wraca do poprzedniej wartości

Zmiana globalna (do restartu serwera)

```
SET GLOBAL innodb_strict_mode = OFF;
```

🔗 uwaga: wymaga uprawnień (np. root). Nowe połączenia będą miały tę wartość

innodb_strict_mode = ON (domyślnie)

```
CREATE TEMPORARY TABLE temp_klienci (  
    id INT  
) ROW_FORMAT=COMPRESSED;
```

```
#4047 - CREATE TEMPORARY TABLE is not allowed with ROW_FORMAT=COMPRESSED or KEY_BLOCK_SIZE.
```

innodb_strict_mode = OFF

```
SHOW VARIABLES LIKE 'innodb_strict_mode';
```

```
SET SESSION innodb_strict_mode = OFF;
```

```
SHOW VARIABLES LIKE 'innodb_strict_mode';  
CREATE TEMPORARY TABLE temp_klienci (  
    id INT  
) ROW_FORMAT=COMPRESSED;
```

Tabela się utworzy, ale **bez kompresji**, dostaniesz ⚠ warning

```
Warning: #1478 InnoDB: ROW_FORMAT=COMPRESSED is ignored for TEMPORARY TABLE.  
Warning: #1478 InnoDB: assuming ROW_FORMAT=DYNAMIC.
```

Przykład kompresji zwykłej tabeli

```
CREATE TABLE klienci_kompresja (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    imie VARCHAR(50),  
    nazwisko VARCHAR(50),  
    email VARCHAR(100)  
) ENGINE=InnoDB  
ROW_FORMAT=COMPRESSED  
KEY_BLOCK_SIZE=8;
```

- ROW_FORMAT=COMPRESSED → włączamy kompresję
- KEY_BLOCK_SIZE=8 → określamy rozmiar strony (np. 8KB)

Kompresja zadziała jeśli używasz **InnoDB**

Kiedy NIE zadziała KOMPRESJA ✗

✗ Przypadek 1 – zły silnik

ENGINE=MyISAM

☞ brak wsparcia dla tej kompresji

✗ Przypadek 2 – brak KEY_BLOCK_SIZE

Czasem:

ROW_FORMAT=COMPRESSED

bez KEY_BLOCK_SIZE może:

- nie działać jak oczekujesz
- albo zostać zignorowane

✗ Przypadek 3 – innodb_strict_mode = ON

Jeśli coś jest nie tak:

SET GLOBAL innodb_strict_mode = ON;

☞ dostaniesz ✗ błąd (np. niepoprawna konfiguracja, czyli np bez podania wartości dla opcji KEY_BLOCK_SIZE podczas tworzenia tabeli)

✗ Przypadek 4 – duże kolumny / ograniczenia

Np.:

- za duże wiersze
- dużo TEXT / BLOB

☞ MySQL może:

- odrzucić kompresję
- albo zmienić format

Sprawdzenie czy działa kompresja na tabeli

```
SHOW TABLE STATUS LIKE 'klienci_kompresja';
```

W kolumnie powinna być wartość:

Row_format: Compressed

Przykład bez strict mode

```
SET GLOBAL innodb_strict_mode = OFF;  
CREATE TABLE test_kompresja (  
    id INT  
) ENGINE=InnoDB  
ROW_FORMAT=COMPRESSED;
```

🔗 efekt:

- tabela się utworzy ✓
- ale może:
 - nie być faktycznie skompresowana !
 - dostać warning ⚠

SPRAWDZIAN 4 ostatnie lekcje – mam przepytac